

PROCESSAMENTO DE LINGUAGEM NATURAL · GRAFOS

Análise Textual de Bios de Matchmaking via Grafos

Como transformar texto livre em um grafo de palavras e extrair padrões de interesse

Paola · Daniel · Siqueira · João Lobo

Integrantes e responsabilidades



João Lobo

Estrutura do grafo, filtragem (Jaccard), análise e integração



Paola

Busca em largura (BFS), centralidade das palavras e geração de dados



Daniel

Cliques (Bron-Kerbosch), grau ponderado, Pré-processamento de texto (NLP)



Siqueira

Union-Find, Kruskal e Prim (árvore geradora máxima)

O tema

Análise textual aplicada a perfis de relacionamento

O problema

Dada uma coleção de biografias de perfis (estilo app de relacionamento), queremos descobrir **padrões nos interesses** que aparecem no texto — sem ler perfil por perfil.

A ideia

Transformar as palavras das bios em um **grafo de coocorrência** e analisar a estrutura desse grafo para responder perguntas sobre o vocabulário.

? Quais grupos de interesses andam sempre juntos?

? Quais palavras conectam temas diferentes?

? Quais interesses estão em alta no dataset?

? Um perfil é "forte" dentro desse vocabulário?

Visão geral: do texto à análise

Cada etapa transforma os dados um pouco mais — vamos ver uma por uma



Etapa 1 — Entrada

1

Tudo começa com as biografias em texto livre

Por quê? É o dado bruto do problema — perfis descritos em linguagem natural, do jeito que uma pessoa escreveria.

Entrada: biografias dos perfis

#1 "Amo música e shows de rock"

#2 "Adoro música e viagens"

#3 "Viagens e fotografia são tudo"

#4 "Rock, shows e fotografia"

Usaremos estas 4 bios como exemplo didático ao longo da explicação

Etapa 2 — Pré-processamento (NLP)

2

Limpamos e normalizamos cada bio antes de montar o grafo

Por quê? Sem isso, "música" e "músicas" virariam vértices diferentes e palavras vazias (de, e, que) poluiriam o grafo.

Pré-processamento (NLP)

Exemplo: "Amo música e shows de rock"

1. Tokenizar + minúsculas

amo · música · shows · de · rock

2. Remover stopwords

música · shows · rock

3. Normalizar (plural/verbo)

música · show · rock

Bibliotecas externas (spaCy/NLTK) são permitidas só nesta etapa de texto

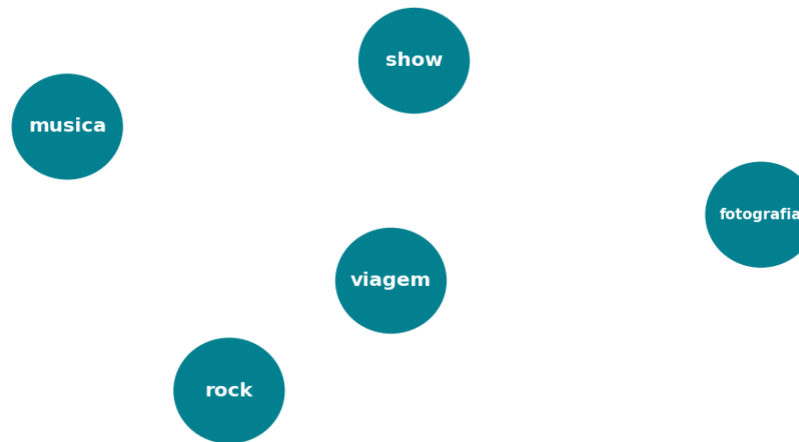
Etapa 3 — Palavras viram vértices

3

Cada palavra única normalizada é um nó do grafo

Por quê? Representar interesses como vértices nos deixa aplicar algoritmos de grafo para achar padrões que não dá pra ver lendo texto.

Cada palavra vira um vértice



5 palavras únicas → 5 vértices (ainda sem ligações)

Ainda não há ligações — só o conjunto de palavras do vocabulário

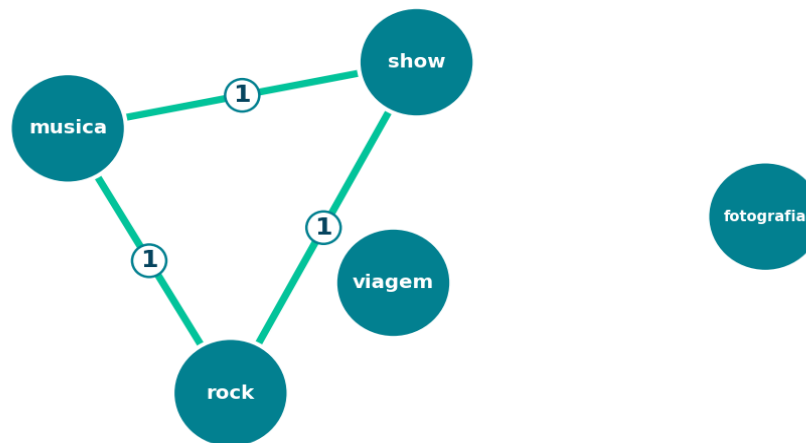
Etapa 4 — Construindo as arestas

4

Palavras que aparecem na mesma bio são ligadas — começamos pela bio #1

Por quê? Coocorrência é o sinal de que dois interesses combinam: se aparecem juntos numa bio, têm relação.

Adicionando a bio #1



#1: "Amo música e shows de rock"

palavras na mesma bio recebem uma aresta (peso += 1)

Bio #1 liga música–show, música–rock e show–rock

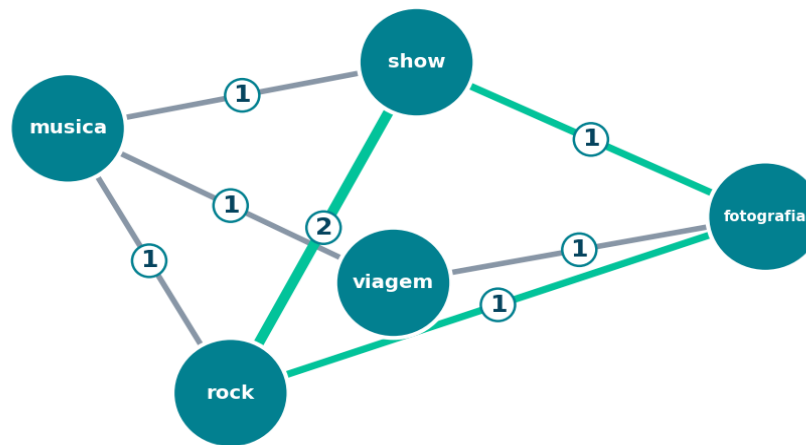
Etapa 4 — As arestas vão se acumulando

4

Cada nova bio adiciona ligações; pares repetidos aumentam o peso

Por quê? O peso transforma "apareceram juntos" em "quão forte é essa relação" — quanto mais bios, mais forte o vínculo.

Adicionando a bio #4



#4: "Rock, shows e fotografia"

palavras na mesma bio recebem uma aresta (peso += 1)

Quando um par reaparece, o peso da aresta aumenta (ex: show–rock vira 2)

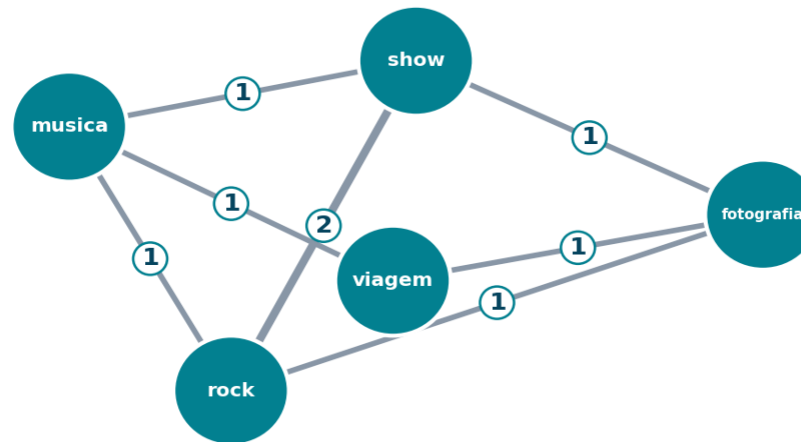
Etapa 5 — Grafo de coocorrência completo

5

Depois de processar todas as bios, temos o grafo com pesos

Por quê? Agora todo o vocabulário das bios virou uma estrutura única que podemos analisar de forma matemática.

Grafo de coocorrência completo



peso da aresta = em quantas bios as duas palavras apareceram juntas

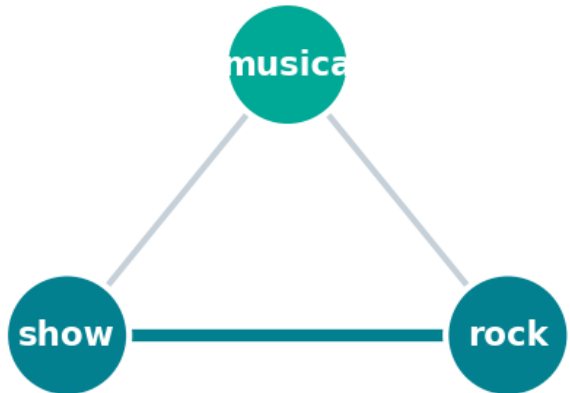
O peso de cada aresta = em quantas bios aquelas duas palavras apareceram juntas

Etapa 6 — Filtragem com Jaccard

6

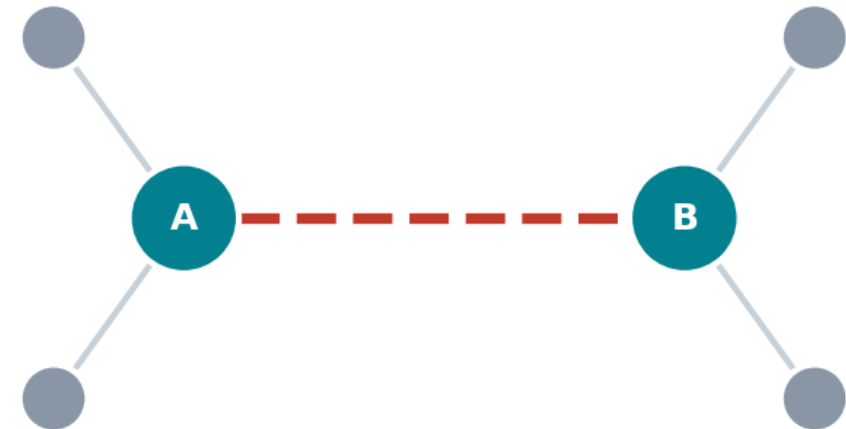
O índice de Jaccard mede quantos vizinhos dois nós têm em comum

Por quê? Algumas palavras coocorrem por acaso. O Jaccard separa relação real (vizinhos em comum) de ruído, limpando o grafo antes das análises.



show e rock têm um vizinho em comum (música)

Jaccard alto → aresta com apoio → mantida



A e B não compartilham nenhum vizinho

Jaccard baixo + peso baixo → candidata a remoção

No dataset real de 50 bios, o filtro removeu cerca de 6% das arestas mais fracas

COM O GRAFO PRONTO E FILTRADO

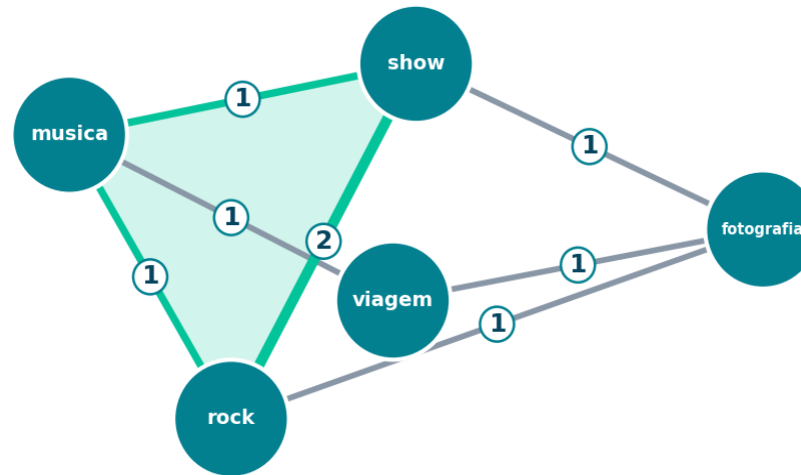
Agora vêm as análises

4 análises textuais + validação técnica dos algoritmos

Análise 1 — Cliques (nichos de interesse)

Grupos de palavras todas ligadas entre si = um tema coeso

Por quê? Cliques revelam quais interesses formam "pacotes" que aparecem sempre juntos — os nichos naturais do dataset.



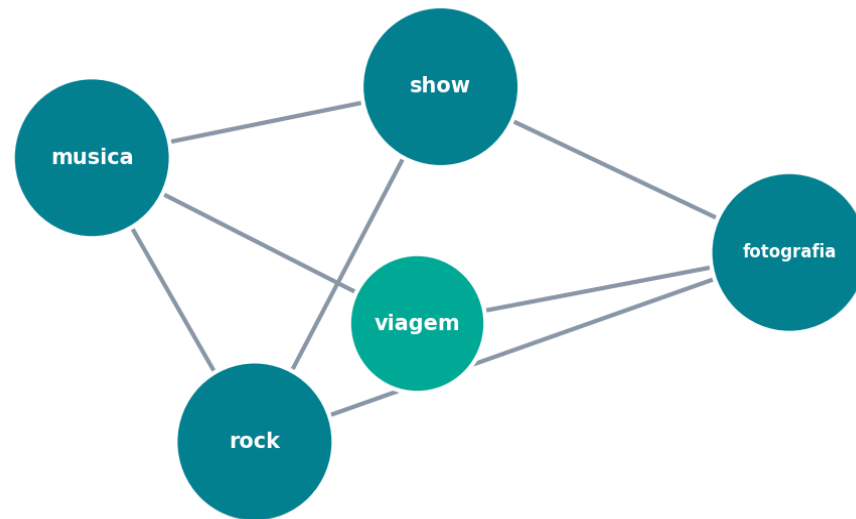
{música, show, rock} = nicho coeso (todas ligadas entre si)

No código real foram encontrados 56 cliques; o mais forte: {apaixonado, hobby, música}

Análise 2 — Palavras que conectam temas

Algumas palavras estão "perto" de todas as outras no grafo (centralidade via BFS)

Por quê? Identificar essas palavras-ponte mostra quais interesses são "universais" e ligam grupos que, sozinhos, não se fariam.



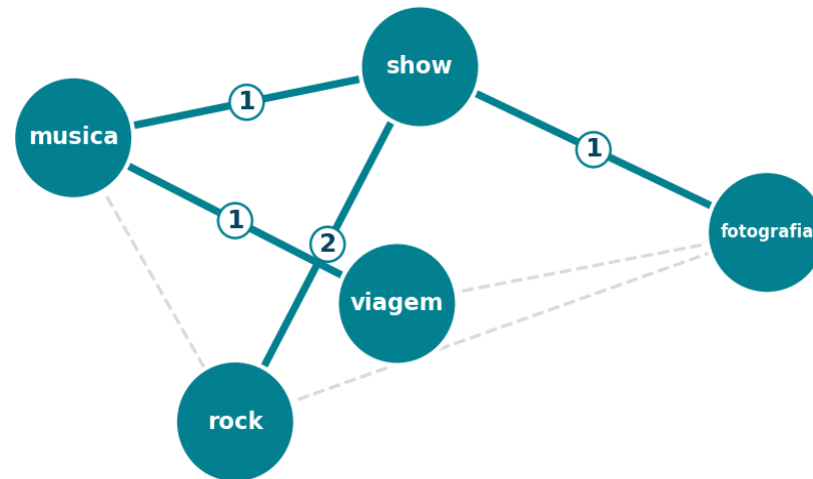
palavras maiores = mais centrais (conectam mais temas)

No dataset real, as que mais conectam temas foram: favorito, música, apaixonado, novo

Análise 3 — Árvore Geradora Máxima

A árvore de máximo peso conecta tudo usando só as ligações mais fortes

Por quê? Um grafo grande tem ligações demais para enxergar. A árvore resume tudo numa árvore com só o essencial.



linhas cheias = ligações mais fortes (sem ciclo) · tracejadas = descartadas

Mostra a "espinha dorsal" do vocabulário: as relações mais importantes, sem redundância

Análise 4 — Palavras em tendência

Grau ponderado: soma dos pesos das arestas de cada palavra

O que mede

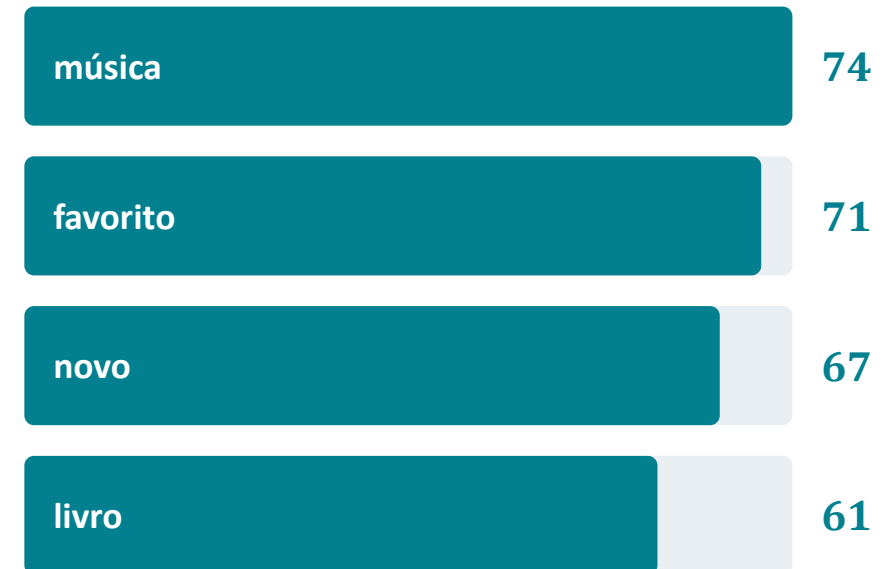
Uma palavra com grau ponderado alto aparece com **muitas outras palavras e com alta frequência** — é um interesse "que está bombando" no dataset.

Dois sub-tipos

- **Nicho intenso:** poucas conexões, mas fortes
- **Interesse genérico:** muitas conexões, mas fracas

Por quê? Mostra quais interesses dominam o vocabulário — e separa os que são realmente fortes dos que só aparecem muito.

Top palavras (dataset real)



Análise 5 — Força do perfil (popularidade)

Junta as análises anteriores para avaliar cada bio individualmente

A pergunta: um conjunto de palavras gera um perfil **popular ou fraco**?

Cobertura de clique

quanto da bio está
em um nicho forte

Centralidade

quão central são
as palavras dela

Grau ponderado

quão "em alta" são
as palavras dela

$$\text{força} = \text{média} (\text{cobertura} \cdot \text{centralidade} \cdot \text{grau})$$

Resposta à pergunta dos cliques:

um clique forte gera perfil popular — mas só para quem cobre bem aquele clique. Cobrir o clique pela metade não traz o mesmo benefício.

Cruzamento entre análises

O insight mais rico vem de comparar os resultados entre si

**Árvore Geradora Máxima ×
Centralidade**

As palavras "hub" da árvore são as mesmas mais centrais no BFS?

Se sim, o esqueleto reflete bem a centralidade real.

Tendência × Cliques

As palavras populares pertencem a nichos coesos?

Distingue interesse popular de interesse genérico.

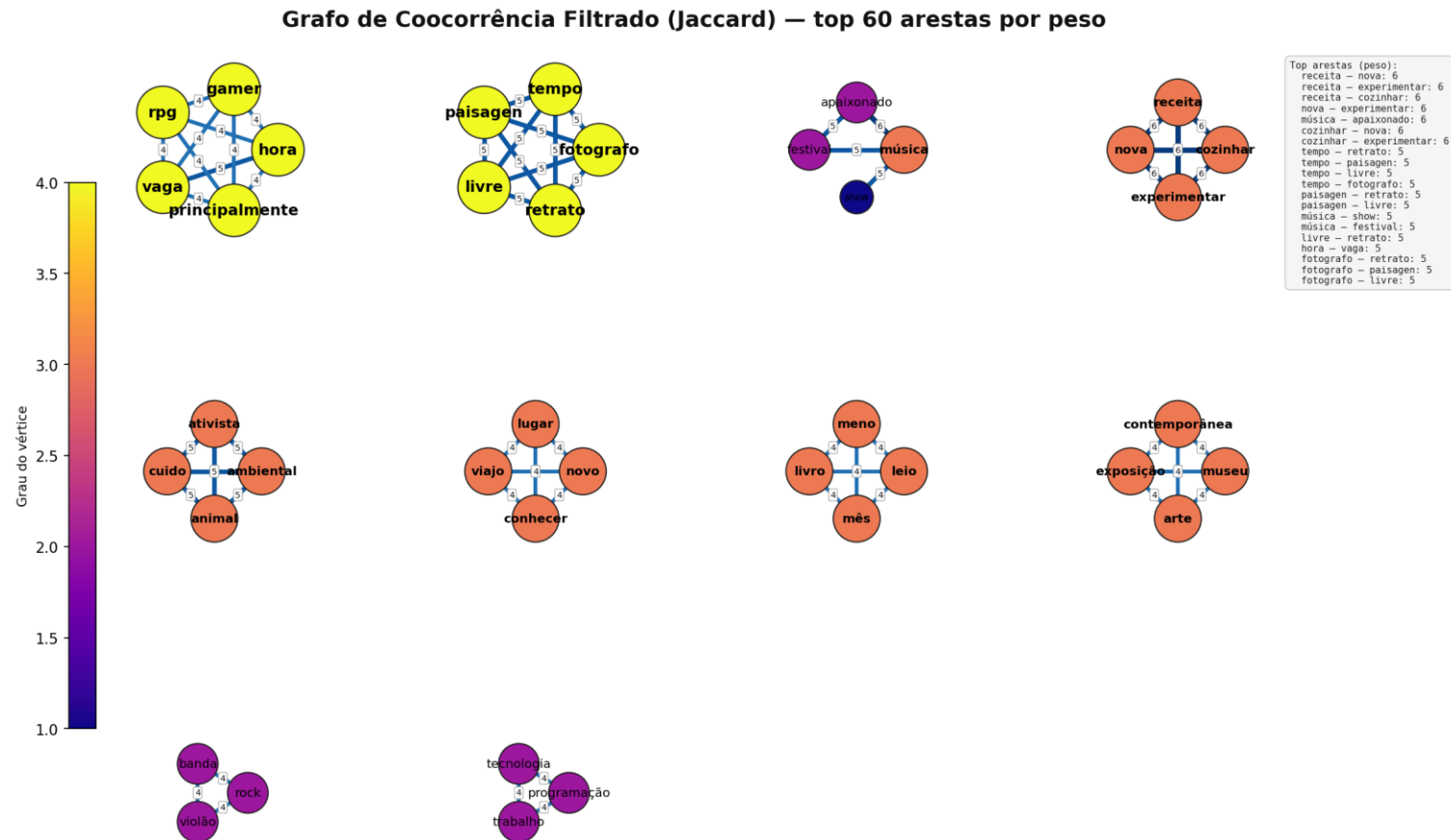
Centralidade × Tendência

Ser central é o mesmo que ser popular?

Mostra se popularidade e conexão andam juntas.

Resultado real — grafo de coocorrência

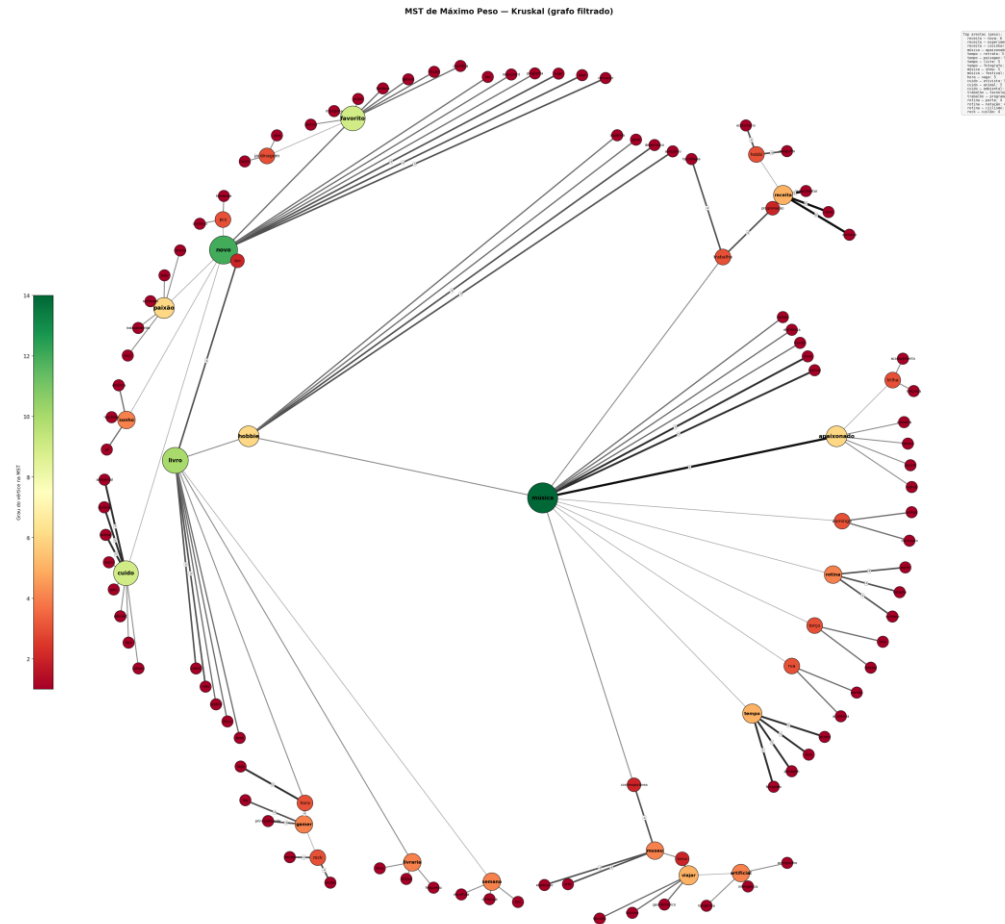
Saída do código com 50 bios: clusters de interesse coloridos por grau



Resultado real — Árvore Geradora Máxima (Kruskal)

R

A árvore geradora máxima do vocabulário real, com "música" no centro



Layout radial: a raiz é a palavra mais conectada, ramos saem para os nichos

Validação técnica — Kruskal vs Prim

Não é análise de dados: é prova de que os algoritmos estão corretos

Por que comparar?

Kruskal e Prim resolvem o mesmo problema por caminhos diferentes. Se ambos chegam no **mesmo peso total**, isso confirma que as duas implementações estão corretas.

Também comparamos o **tempo de execução** em grafos de tamanhos diferentes, para ver na prática como cada um escala.

Vértices	Kruskal	Prim
30	0,00019 s	0,00010 s
61	0,00052 s	0,00033 s
92	0,00091 s	0,00047 s
123	0,00118 s	0,00062 s

Prim com heap foi mais rápido em todos os tamanhos testados

O que foi implementado do zero

Grafo + estruturas e algoritmos, sem bibliotecas prontas para o núcleo



Grafo de coocorrência

lista de adjacência com pesos



BFS

distância e centralidade



Bron-Kerbosch

cliques maximais



Union-Find

estrutura extra (usada no Kruskal)



Kruskal & Prim

árvore geradora de máximo peso



Jaccard

filtragem estrutural do grafo

Como o LLM foi usado

Transparência sobre onde a IA ajudou no desenvolvimento



Geração das bios fictícias *(com LLM)*

Criação do banco de frases por tema para os dados de teste



Apoio no planejamento *(com LLM)*

Discussão da modelagem do grafo e organização das etapas



Revisão e depuração *(com LLM)*

Ajuda para encontrar e corrigir problemas no código



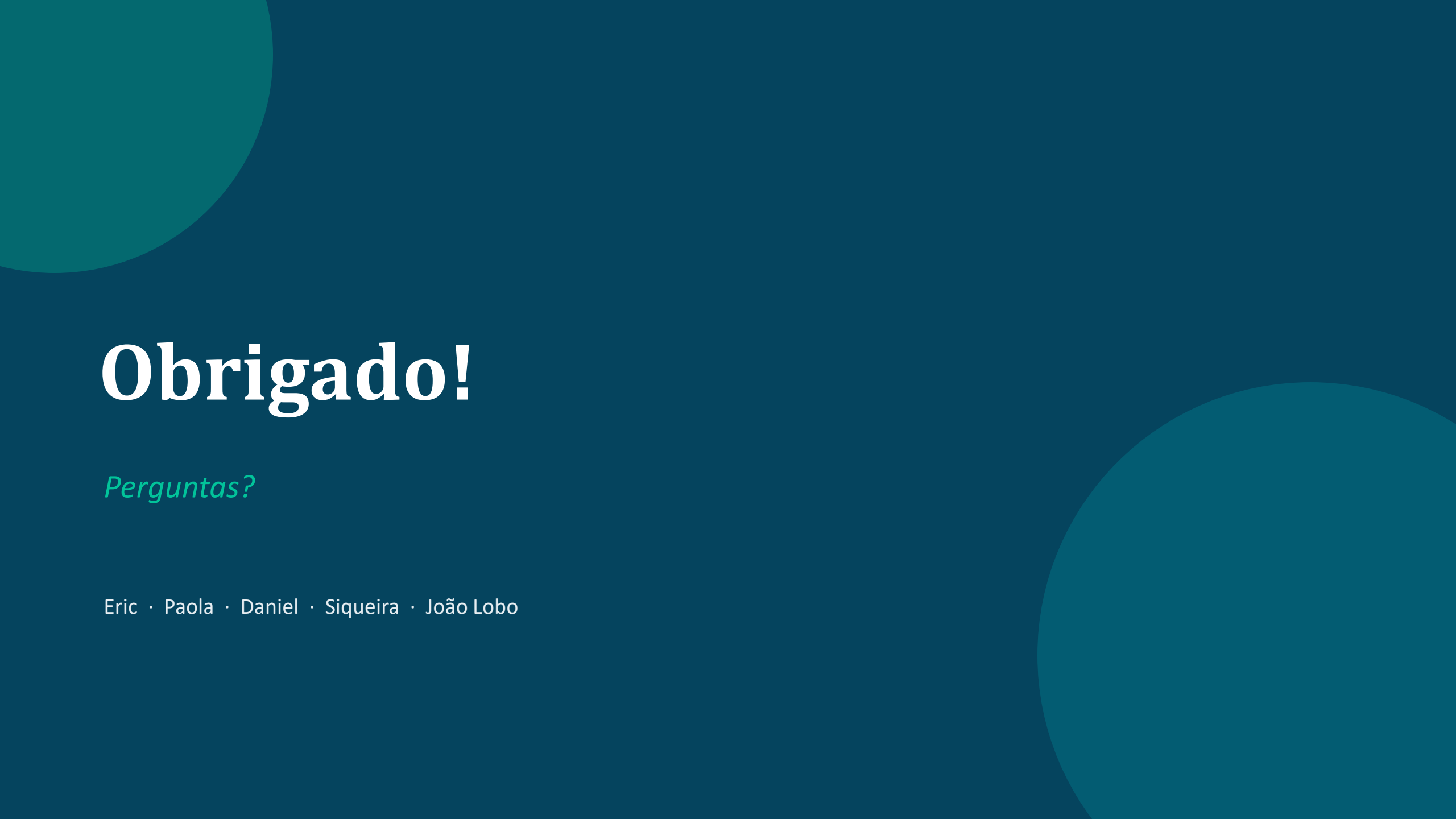
Algoritmos de grafo *(feito pelo grupo)*

Implementados pelo grupo — BFS, cliques, Kruskal, Prim, Jaccard

CONCLUSÃO

O que conseguimos mostrar

- Texto livre vira um grafo de palavras com pesos de coocorrência
- A estrutura do grafo revela nichos (cliques), palavras-ponte (centralidade) e tendências (grau)
- A Árvore Geradora Máxima mostra o esqueleto semântico do vocabulário
- A força de perfil responde se um conjunto de palavras é popular
- Todos os algoritmos de grafo foram implementados pelo grupo



Obrigado!

Perguntas?

Eric · Paola · Daniel · Siqueira · João Lobo